МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
А.С. Кот
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1(порождающий паттерн):
10) Заводы по производству автомобилей. Реализовать возможность создавать автомобили различных типов на различных заводах.

```
from abc import ABC, abstractmethod


# Абстрактные продукты
class Engine(ABC):
    @abstractmethod
    def get_specs(self):
        pass


class Body(ABC):
    @abstractmethod
    def get_type(self):
        pass


# Конкретные продукты для Toyota
class ToyotaEngine(Engine):
    def get_specs(self):
        return "Toyota 2.0L VVT-i"


class ToyotaSedanBody(Body):
    def get_type(self):
        return "Sedan (Toyota Camry style)"


class ToyotaSUVBody(Body):
    def get_type(self):
        return "SUV (Toyota RAV4 style)"


# Конкретные продукты для BMW
class BMWEngine(Engine):
    def get_specs(self):
        return "BMW 3.0L TwinPower Turbo"


class BMWSedanBody(Body):
    def get_type(self):
        return "Sedan (BMW 3 Series style)"


class BMWSUVBody(Body):
    def get_type(self):
        return "SUV (BMW X5 style)"


# Абстрактная фабрика
class CarFactory(ABC):
    @abstractmethod
    def create_engine(self) -> Engine:
        pass

    @abstractmethod
```

```
    def create_body(self) -> Body:
        pass

    def assemble_car(self, car_type: str):
        engine = self.create_engine()
        body = self.create_body() if car_type == "Sedan" else self.create_suv_body()

        print(f"[BUILD] Starting car assembly at {self.__class__.__name__}")
        print(f"[INFO] Engine installed: {engine.get_specs()}")
        print(f"[INFO] Body installed: {body.get_type()}")
        print(
            f"[RESULT] Car successfully created: {self.__class__.__name__}
{car_type}\n"
        )
        return f"{self.__class__.__name__} {car_type}"


# Конкретные фабрики
class ToyotaFactory(CarFactory):
    def create_engine(self):
        return ToyotaEngine()

    def create_body(self):
        return ToyotaSedanBody()

    def create_suv_body(self):
        return ToyotaSUVBody()


class BMWFactory(CarFactory):
    def create_engine(self):
        return BMWEngine()

    def create_body(self):
        return BMWSedanBody()

    def create_suv_body(self):
        return BMWSUVBody()


# Клиентский код
def produce_car(factory: CarFactory, model: str):
    print(f"=== Order received: {factory.__class__.__name__} {model} ===")
    factory.assemble_car(model)


if __name__ == "__main__":
    toyota_factory = ToyotaFactory()
    bmw_factory = BMWFactory()

    produce_car(toyota_factory, "Sedan")
    produce_car(toyota_factory, "SUV")
    produce_car(bmw_factory, "Sedan")
    produce_car(bmw_factory, "SUV")
```

```
=== Order received: ToyotaFactory Sedan ===
[BUILD] Starting car assembly at ToyotaFactory
[INFO] Engine installed: Toyota 2.0L VVT-i
[INFO] Body installed: Sedan (Toyota Camry style)
[RESULT] Car successfully created: ToyotaFactory Sedan

=== Order received: ToyotaFactory SUV ===
[BUILD] Starting car assembly at ToyotaFactory
[INFO] Engine installed: Toyota 2.0L VVT-i
[INFO] Body installed: SUV (Toyota RAV4 style)
[RESULT] Car successfully created: ToyotaFactory SUV

=== Order received: BMWFactory Sedan ===
[BUILD] Starting car assembly at BMWFactory
[INFO] Engine installed: BMW 3.0L TwinPower Turbo
[INFO] Body installed: Sedan (BMW 3 Series style)
[RESULT] Car successfully created: BMWFactory Sedan

=== Order received: BMWFactory SUV ===
[BUILD] Starting car assembly at BMWFactory
[INFO] Engine installed: BMW 3.0L TwinPower Turbo
[INFO] Body installed: SUV (BMW X5 style)
[RESULT] Car successfully created: BMWFactory SUV
```

Задание 2 (структурный паттерн):

10) Учетная запись покупателя книжного интернет-магазина. Предусмотреть различные уровни учетки в зависимости от активности покупателя. Дополнительные уровни добавляют функциональные возможности и открывают доступ к уникальным предложениям.

```
from abc import ABC, abstractmethod


# Компонент
class Account(ABC):
    @abstractmethod
    def get_benefits(self):
        pass

    @abstractmethod
    def get_discount(self):
        pass


# Конкретный компонент (базовый уровень)
class BaseAccount(Account):
    def __init__(self, name: str):
        self.name = name
        print(f"[INIT] Base account created for: {name}")

    def get_benefits(self):
        return ["Standard catalog access"]

    def get_discount(self):
        return 0.0


# Абстрактный декоратор
class AccountDecorator(Account):
    def __init__(self, account: Account):
        self._account = account
        print(
            f"[DECORATE] Adding new features to: {account.name if hasattr(account,
'name') else 'Account'}"
        )
```

```
    def get_benefits(self):
        return self._account.get_benefits()

    def get_discount(self):
        return self._account.get_discount()


# Конкретные декораторы
class BronzeDecorator(AccountDecorator):
    def get_benefits(self):
        benefits = self._account.get_benefits()
        benefits.append("Free standard shipping")
        return benefits

    def get_discount(self):
        return 5.0


class SilverDecorator(AccountDecorator):
    def get_benefits(self):
        benefits = self._account.get_benefits()
        benefits.append("Priority customer support")
        benefits.append("Exclusive monthly e-book")
        return benefits

    def get_discount(self):
        return 10.0


class GoldDecorator(AccountDecorator):
    def get_benefits(self):
        benefits = self._account.get_benefits()
        benefits.append("VIP early access to new releases")
        benefits.append("Free express shipping")
        benefits.append("Personal book advisor")
        return benefits

    def get_discount(self):
        return 15.0


# Клиентский код
if __name__ == "__main__":
    print("--- Creating customer: John Doe ---")
    user = BaseAccount("John Doe")

    print("\n--- Upgrading to Bronze level ---")
    user = BronzeDecorator(user)

    print("\n--- Upgrading to Silver level ---")
    user = SilverDecorator(user)

    print("\n--- Upgrading to Gold level ---")
    user = GoldDecorator(user)

    print("\n=== Final Account Status ===")
    print(f"Benefits: {', '.join(user.get_benefits())}")
    print(f"Discount: {user.get_discount()}%")
```

```
--- Creating customer: John Doe ---
[INIT] Base account created for: John Doe

--- Upgrading to Bronze level ---
[DECORATE] Adding new features to: John Doe

--- Upgrading to Silver level ---
[DECORATE] Adding new features to: Account

--- Upgrading to Gold level ---
[DECORATE] Adding new features to: Account

=== Final Account Status ===
Benefits: Standard catalog access, Free standard
 shipping, Priority customer support, Exclusive
monthly e-book, VIP early access to new releases
, Free express shipping, Personal book advisor
Discount: 15.0%
Press any key to continue . . .
```

Задание 3(поведенческий паттерн)

10) Проект «Банкомат». Предусмотреть выполнение основных операций (ввод пин-кода, снятие суммы, завершение работы) и наличие различных режимов работы (ожидание, аутентификация, выполнение операции, блокировка – если нет денег). Атрибуты: общая сумма денег в банкомате, ID.

```
from abc import ABC, abstractmethod


class ATM:
    def __init__(self, atm_id: str, initial_balance: float):
        self.atm_id = atm_id
        self.total_cash = initial_balance
        self._state = None
        self._authenticated_user = None

        # Initialize with Idle state
        self.change_state(IdleState())
        print(
            f"[ATM:{atm_id}] System started. Balance: ${initial_balance:.2f}. State:
IDLE"
        )

    def change_state(self, new_state):
        self._state = new_state
        self._state.set_atm(self)
        print(f"[ATM:{atm_id}] -> State changed to: {new_state.__class__.__name__}")

    def insert_card(self):
        self._state.insert_card()

    def enter_pin(self, pin):
        self._state.enter_pin(pin)

    def withdraw(self, amount):
        self._state.withdraw(amount)

    def end_session(self):
        self._state.end_session()

    # Getters and setters for context data
    def get_cash(self):
```

```
        return self.total_cash

    def set_cash(self, amount):
        self.total_cash = amount
        print(f"[ATM:{self.atm_id}] Cash balance updated: ${self.total_cash:.2f}")

    def is_cash_available(self, amount):
        return self.total_cash >= amount

    def authenticate_user(self, pin):
        # Dummy authentication: any PIN except 0000 is valid
        if pin == "0000":
            print("[AUTH] Invalid PIN code.")
            return False
        self._authenticated_user = "Customer"
        print("[AUTH] User authenticated successfully.")
        return True

    def clear_auth(self):
        self._authenticated_user = None
        print("[AUTH] User logged out.")


# Абстрактное состояние
class State(ABC):
    def __init__(self):
        self._atm = None

    def set_atm(self, atm):
        self._atm = atm

    @abstractmethod
    def insert_card(self):
        pass

    @abstractmethod
    def enter_pin(self, pin):
        pass

    @abstractmethod
    def withdraw(self, amount):
        pass

    @abstractmethod
    def end_session(self):
        pass


# Конкретные состояния
class IdleState(State):
    def insert_card(self):
        print("[ACTION] Card inserted. Please enter PIN.")
        self._atm.change_state(AuthenticatingState())

    def enter_pin(self, pin):
        print("[ERROR] No card inserted. Please insert card first.")

    def withdraw(self, amount):
        print("[ERROR] No session active. Please insert card.")

    def end_session(self):
        print("[INFO] No active session to end.")


class AuthenticatingState(State):
```

```
    def insert_card(self):
        print("[WARN] Card already detected. Please enter PIN.")

    def enter_pin(self, pin):
        if self._atm.authenticate_user(pin):
            print("[ACTION] Authentication successful. Select operation.")
            self._atm.change_state(OperationState())
        else:
            print("[ERROR] Authentication failed. Ejecting card.")
            self._atm.change_state(IdleState())

    def withdraw(self, amount):
        print("[ERROR] Please complete authentication first.")

    def end_session(self):
        print("[INFO] Session cancelled by user. Ejecting card.")
        self._atm.change_state(IdleState())


class OperationState(State):
    def insert_card(self):
        print("[INFO] Card already in session.")

    def enter_pin(self, pin):
        print("[INFO] Already authenticated.")

    def withdraw(self, amount):
        if amount <= 0:
            print("[ERROR] Invalid amount.")
            return

        if not self._atm.is_cash_available(amount):
            print(
                f"[ERROR] Insufficient funds in ATM. Available:
${self._atm.get_cash():.2f}"
            )
            print("[BLOCK] ATM is out of cash. Entering BLOCKED mode.")
            self._atm.change_state(BlockedState())
            return

        print(f"[TRANSACTION] Dispensing ${amount:.2f}...")
        self._atm.set_cash(self._atm.get_cash() - amount)
        print(
            f"[SUCCESS] Please take your cash. Remaining balance in ATM:
${self._atm.get_cash():.2f}"
        )
        self.end_session()

    def end_session(self):
        print("[SESSION] Ending session. Thank you for using our ATM.")
        self._atm.clear_auth()
        self._atm.change_state(IdleState())


class BlockedState(State):
    def insert_card(self):
        print("[BLOCKED] ATM is out of service. Please contact support.")

    def enter_pin(self, pin):
        print("[BLOCKED] Operation unavailable.")

    def withdraw(self, amount):
        print("[BLOCKED] No cash available.")

    def end_session(self):
```

```
        print("[BLOCKED] Forced session end.")
        self._atm.change_state(IdleState())


# Клиентский код
if __name__ == "__main__":
    atm_id = "ATM-001"
    atm = ATM(atm_id, 500.00)

    print("\n=== Scenario 1: Normal operation ===")
    atm.insert_card()
    atm.enter_pin("1234")
    atm.withdraw(200.00)

    print("\n=== Scenario 2: Wrong PIN ===")
    atm.insert_card()
    atm.enter_pin("0000")

    print("\n=== Scenario 3: Withdraw more than available (trigger BLOCKED) ===")
    atm.insert_card()
    atm.enter_pin("1111")
    atm.withdraw(600.00)

    print("\n=== Scenario 4: Try to use blocked ATM ===")
    atm.insert_card()
```

```
[ATM:ATM-001] -> State changed to: IdleState
[ATM:ATM-001] System started. Balance: $500.00. State: IDLE

=== Scenario 1: Normal operation ===
[ACTION] Card inserted. Please enter PIN.
[ATM:ATM-001] -> State changed to: AuthenticatingState
[AUTH] User authenticated successfully.
[ACTION] Authentication successful. Select operation.
[ATM:ATM-001] -> State changed to: OperationState
[TRANSACTION] Dispensing $200.00...
[ATM:ATM-001] Cash balance updated: $300.00
[SUCCESS] Please take your cash. Remaining balance in ATM: $300.00
[SESSION] Ending session. Thank you for using our ATM.
[AUTH] User logged out.
[ATM:ATM-001] -> State changed to: IdleState

=== Scenario 2: Wrong PIN ===
[ACTION] Card inserted. Please enter PIN.
[ATM:ATM-001] -> State changed to: AuthenticatingState
[AUTH] Invalid PIN code.
[ERROR] Authentication failed. Ejecting card.
[ATM:ATM-001] -> State changed to: IdleState

=== Scenario 3: Withdraw more than available (trigger BLOCKED) ===
[ACTION] Card inserted. Please enter PIN.
[ATM:ATM-001] -> State changed to: AuthenticatingState
[AUTH] User authenticated successfully.
[ACTION] Authentication successful. Select operation.
[ATM:ATM-001] -> State changed to: OperationState
[ERROR] Insufficient funds in ATM. Available: $300.00
[BLOCK] ATM is out of cash. Entering BLOCKED mode.
[ATM:ATM-001] -> State changed to: BlockedState

=== Scenario 4: Try to use blocked ATM ===
[BLOCKED] ATM is out of service. Please contact support.
```

Вывод: были отработаны навыки применения паттернов проектирования при решении практических задач с использованием языка Python.